Lecture 17: Remoting I

Objectives:

- Learn the basics of remoting
- Learn the elements required to create a remoting application.

1. Remoting Basics

1.1 What is Remoting?

Remoting is the ability to call a method of an object that is on a different application domain as if it were in the current application domain.

The two application domains can be on the same computer on different computers in a network.

1.2 Types of Remoting

There are two ways an object in one application domain can be made available to another application domain.

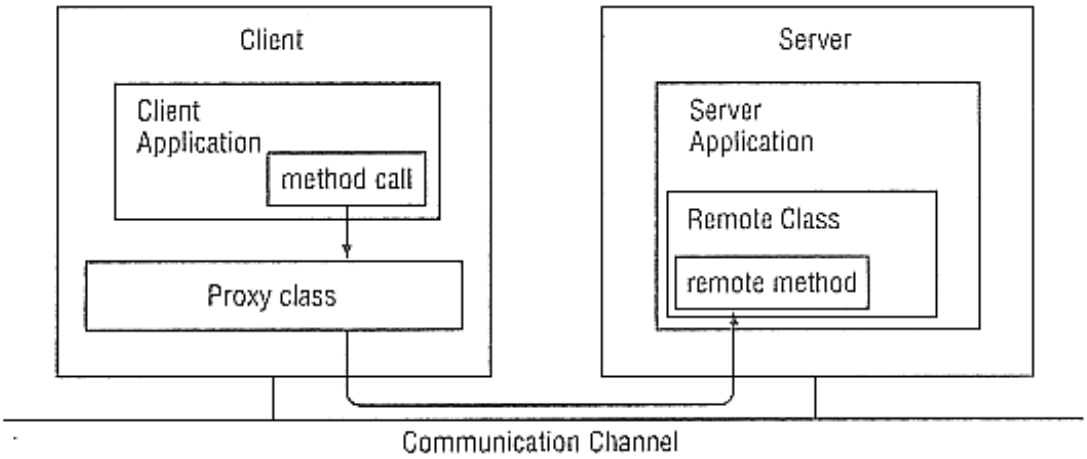
The first option involves serializing the object, transport it to the other domain using streams. The object is then de-serialized and used at the other side. This version is called **Marshall by value (MBV).**

MBV should be considered only if the object does not depend on any data in its original domain.

The second type of remoting is called **Marshall by reference (MBR).**
In this case, the client communicates with the remotable object through a proxy, but the object remains in its application domain.

To the client, the proxy appears as if it is the actual object. However, for each call made to the proxy, the proxy passes the call to the remote object using a communication channel, obtain a result from the remote object and pass same to the client.

The following figure shows the set-up for MBR.



2. Creating an MBR Remoting application.

To create an MBR remoting application, the following elements are required.

- A remotable class
- A remoting server
- A remoting client

2.1 Remotable class

This is a class whose methods can be accessed from another application domain. To construct such a class, all you need to do is to extend the **MashalByRefObject** class, which is in the **System** namespace.

The following is an example of a remotable class.

```
using System;

public class MathClass:MarshalByRefObject {
      public double Add(double a, double b) {
            return a + b;
      }
      public double Subtract(double a, double b) {
            return a - b;
      }
      public double Multiply(double a, double b) {
            return a * b;
      }
      public double Divide(double a, double b) {
            if (b == 0)
                  return 0;
            else
                  return a/b;
      }
}
```

Note that you need to compile this class into a DLL.

2.2 Remoting Server

This is an application that a client connects to in order to gain access to the method of the remotable class. Such an application must do three things as follows:

a. Create a communication channel: This can be done with either the **TcpChannel** class of **System.Runtime.Remoting.Channels.Tcp** namespace, or using the **HttpChannel** class of **System.Runtime.Remoting.Channels.Http** namespace.

b. Register the communication channel created in (a) with the remoting channel services. This is done by passing the channel to the static method, ***RegisterChannel***, of the **ChannelServices** class, which is in the **System.Runtime.Remoting.Channels** namespace.

c. Register the remotable class with the remoting server. This is done by using the static method, ***RegisterWellKnownServiceType*** of the **RemotingConfiguration** class, of the **System.Runtime.Remoting** namespace.

This method takes three arguments:

- The *type* of the remotable class
- A *URI* identifier for the class, and
- Object creation *mode*.

Possible modes are: **SingleCall** and **Singleton**, both of which are fields of the **WellKnownObjectMode** class.

- **SingleCall** means a separate instance of the remotable class will be created for each call to the remotable class.

- **Singleton** mode means, a single instance will be used for different calls for all clients. Singleton is useful if you wish to retain the state across different calls.

The following example creates a remoting server for our MathClass:

```
using System;
using System.Runtime.Remoting;
using System.Runtime.Remoting.Channels;
using System.Runtime.Remoting.Channels.Http;

public class MathServer {
     public static void Main() {
          HttpChannel channel = new HttpChannel(9095);
          ChannelServices.RegisterChannel(channel);

     RemotingConfiguration.RegisterWellKnownServiceType(
                    typeof(MathClass),  //type of Remotable
class
                      "MyMathServer",     //URI
                    WellKnownObjectMode.SingleCall); //Mode
          Console.WriteLine("Press <enter> to exit...");
          Console.ReadLine();
     }
}
```

Note here that you need to store the **MathClass.dll** in the same folder as the MathServer.exe

2.3 Remoting Client

This is the application that is used to access the methods of the remotable class through the remoting server.

Here again, there are three things that the client class must do to communicate with the remotable class:

a. Create a channel. This must be of the same type as that of the remoting server.

b. Register the communication channel created in (a) with the remoting channel services.

c. Creating an instance of the proxy class. The proxy class is like an alias to the remotable class, so all calls to the remotable class are made through the proxy class. Here we have two options:

   i. We use the ***RegisterWellKnownClientType*** of the **RemotingConfiguration** class. This takes the type of the

remote class and its URI as arguments. Example:

```
RemotingConfiguration.RegisterWellKnownClientType(
        typeof(MathClass),  //type of Remotable class
    "http://localhost:9095/MyMathServer");   //server URI

  MathClass math = new MathClass();
```

ii. Alternatively, we can use the ***getObject*** method of the **Activator** class, which is in the System namespace. Example:

```
MathClass math = (MathClass) Activator.GetObject(
typeof(MathClass),
"http://localhost:9095/MyMathServer");
```

Note that in both options (i) and (ii), we need the type of the **MathClass** for the statements to compile.

If we have the same MathClass.dll at the client side, this will work. But then why do we need the remotable class if we have it locally at the client side?

Well, the MathClass.dll at the client side does not need to have all the implementation details, just the meta data.

In fact we can use a utility called **soapsuds** that comes with .NET SDK to generate such a dll from the remote server using the following command:

```
soapsuds –url:http://localhost:9095/MyMathServer?wsdl –
oa:MathClass.dll -nowp
```

Alternatively, when designing the MathClass, you can first create an interface containing the methods you wish to make remotable, then write the Math class to implement the interface.

In this case, you provide the client with the interface, which he can then use to create the proxy class.

In the following example, the soapsuds utility is used to generate the MathClass.dll file at the client side and then option (i) is used.

```
using System;
using System.Runtime.Remoting;
using System.Runtime.Remoting.Channels;
using System.Runtime.Remoting.Channels.Http;

public class MathClient {
     public static void Main(string[] args) {
          HttpChannel channel = new HttpChannel();
          ChannelServices.RegisterChannel(channel);

          RemotingConfiguration.RegisterWellKnownClientType (
             typeof (MathClass),            // Remotable class
             "http://localhost:9095/MyMathServer" // URL of remotable
class
          );

          MathClass math = new MathClass();
          if (math == null)
               Console.WriteLine("Could not locate Server");
          else      {
               int a = 10; int b = 5;
               Console.WriteLine("{0} + {1} = {2}", a, b, math.Add(a,
b));
               Console.WriteLine("{0} - {1} = {2}", a, b,
math.Subtract(a, b));
               Console.WriteLine("{0} * {1} = {2}", a, b,
math.Multiply(a, b));
               Console.WriteLine("{0} / {1} = {2}", a, b, math.Divide(a,
b));
               Console.ReadLine();
          }
     }
}
```